

```

%% Projeto de Processamento Digital de Sinais - UFRN
% Método: Sobreposição e Soma (Overlap-Add)
% Prof. Felipe Silveira

clear all; close all; clc;

%% ===== PASSO 1: VALIDAÇÃO =====
disp('===== PASSO 1: VALIDAÇÃO DO ALGORITMO =====');

```

```

===== PASSO 1: VALIDAÇÃO DO ALGORITMO =====

```

```

% Parâmetros para validação
N = 512; % Número de pontos da FFT (pode usar 1024 também)
M_val = 150; % Número de amostras da resposta ao impulso h[n] PARA
VALIDAÇÃO
L = N - M_val + 1; % Comprimento de cada bloco
Lx = round(15.3 * L); % Comprimento do sinal de entrada (~15.3*L)

disp(['Parâmetros de validação: N=', num2str(N), ', M=', num2str(M_val), ', L=',
num2str(L)]);

```

```

Parâmetros de validação: N=512, M=150, L=363

```

```

disp(['Comprimento do sinal de entrada: ', num2str(Lx), ' amostras (≈',
num2str(Lx/L, '%.1f'), '*L)']);

```

```

Comprimento do sinal de entrada: 5554 amostras (≈15.3*L)

```

```

% Gerar sinais aleatórios para teste
x = randn(1, Lx); % Sinal de entrada aleatório
h_val = randn(1, M_val); % Filtro FIR aleatório para validação

% Convolução usando Overlap-Add (nossa implementação)
tic;
y_overlap = overlap_add(x, h_val, N);
tempo_overlap = toc;

% Convolução no tempo (para validação)
tic;
y_conv = conv(x, h_val);
tempo_conv = toc;

% Comparação dos resultados
erro = max(abs(y_overlap - y_conv));
disp(['Erro máximo entre overlap-add e convolução no tempo: ', num2str(erro)]);

```

```

Erro máximo entre overlap-add e convolução no tempo: 4.2633e-14

```

```

disp(['Tempo overlap-add: ', num2str(tempo_overlap), ' s']);

```

Tempo overlap-add: 0.01146 s

```
disp(['Tempo convolução no tempo: ', num2str(tempo_conv), ' s']);
```

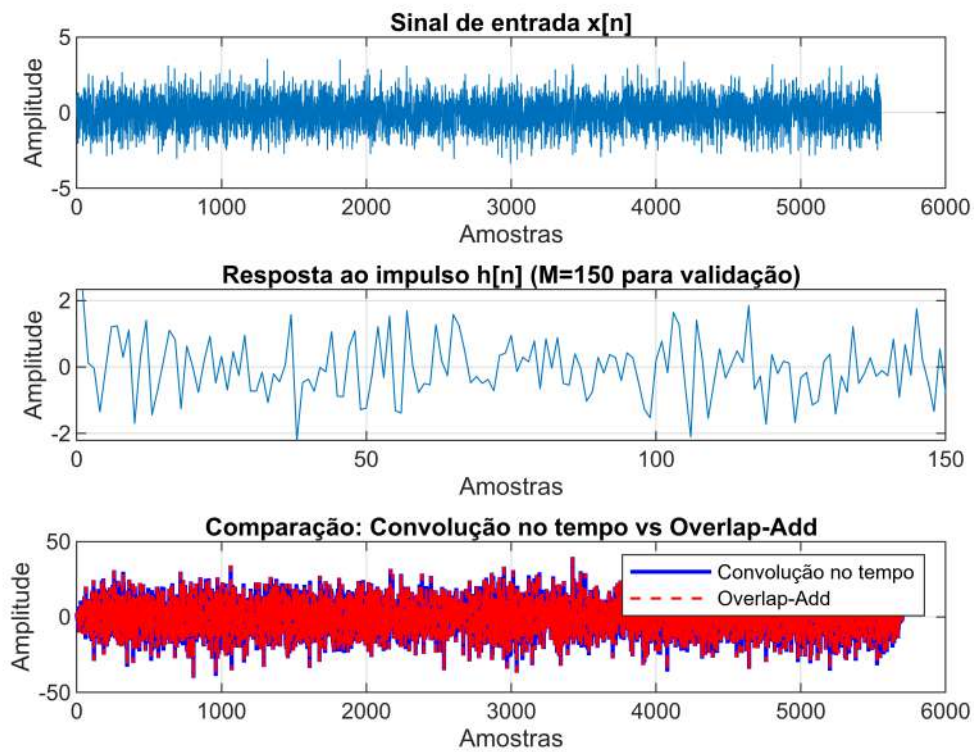
Tempo convolução no tempo: 0.0060624 s

```
% Plotar resultados da validação
figure('Name', 'Passo 1 - Validação');

subplot(3,1,1);
plot(x);
title('Sinal de entrada x[n]');
xlabel('Amostras'); ylabel('Amplitude');
grid on;

subplot(3,1,2);
plot(h_val);
title('Resposta ao impulso h[n] (M=150 para validação)');
xlabel('Amostras'); ylabel('Amplitude');
grid on;

subplot(3,1,3);
plot(y_conv, 'b', 'LineWidth', 1.5); hold on;
plot(y_overlap, 'r--', 'LineWidth', 1);
title('Comparação: Convolução no tempo vs Overlap-Add');
xlabel('Amostras'); ylabel('Amplitude');
legend('Convolução no tempo', 'Overlap-Add');
grid on;
```



```
%% ===== PASSO 2: FILTRAGEM DE VOZ =====
disp(' ');
```

```
disp('===== PASSO 2: FILTRAGEM DE VOZ =====');
```

```
===== PASSO 2: FILTRAGEM DE VOZ =====
```

```
% Carregar áudio existente
disp('Carregando arquivo de áudio...');
```

```
Carregando arquivo de áudio...
```

```
arquivo_audio = 'Audio_Hanna2.m4a';
```

```
% Ler arquivo de áudio/vídeo
[audio_entrada, Fs] = audioread(arquivo_audio);
```

```
% Verificar se é estéreo e converter para mono se necessário
if size(audio_entrada, 2) == 2
    audio_entrada = mean(audio_entrada, 2); % Converter para mono
    disp('Áudio convertido de estéreo para mono');
end
```

```
% Transpor para vetor linha
```

```
audio_entrada = audio_entrada';

duracao = length(audio_entrada) / Fs;
disp(['Áudio carregado com sucesso!']);
```

Áudio carregado com sucesso!

```
disp(['Duração: ', num2str(duracao), ' segundos']);
```

Duração: 13.2265 segundos

```
disp(['Frequência de amostragem: ', num2str(Fs), ' Hz']);
```

Frequência de amostragem: 48000 Hz

```
% Salvar áudio de entrada em formato WAV
audiowrite('audio_entrada.wav', audio_entrada, Fs);
disp('Áudio de entrada salvo em: audio_entrada.wav');
```

Áudio de entrada salvo em: audio_entrada.wav

```
% Projeto do filtro FIR (MESMO FILTRO DO PROJETO DA UII)
% Filtro Passa-Baixas com Janela de Blackman
disp('Projetando filtro FIR com janela de Blackman...');
```

Projetando filtro FIR com janela de Blackman...

```
% Parâmetros do Filtro (do projeto UII)
M = 275; % Ordem do Filtro
alfa = M / 2; % Atraso do Filtro
Wc = 0.813 * pi; % Frequência de Corte Normalizada (radianos)
fc = Wc * Fs / (2 * pi); % Frequência de corte em Hz
```

```
% Vetor de tempo discreto
n = 0:M;
```

```
% 1. Cálculo da Resposta ao Impulso Ideal (hd[n])
hd_n = sin(Wc * (n - alfa)) ./ (pi * (n - alfa));
hd_n(n == alfa) = Wc / pi; % Corrige a indeterminação em n = alfa
```

```
% 2. Função Janela de Blackman
w_n = 0.42 - 0.5 * cos(2*pi*n / M) + 0.08 * cos(4*pi*n / M);
```

```
% 3. Resposta ao Impulso Final do Filtro (h[n])
h_filtro = hd_n .* w_n;
```

```
disp(['Tipo de filtro: Passa-Baixas com Janela de Blackman']);
```

Tipo de filtro: Passa-Baixas com Janela de Blackman

```
disp(['Ordem do filtro (M): ', num2str(M)]);
```

Ordem do filtro (M): 275

```
disp(['Frequência de corte: ', num2str(fc), ' Hz']);
```

Frequência de corte: 19512 Hz

```
disp(['Frequência de amostragem: ', num2str(Fs), ' Hz']);
```

Frequência de amostragem: 48000 Hz

```
% Aplicar filtragem usando Overlap-Add  
disp('Aplicando filtro usando Overlap-Add...');
```

Aplicando filtro usando Overlap-Add...

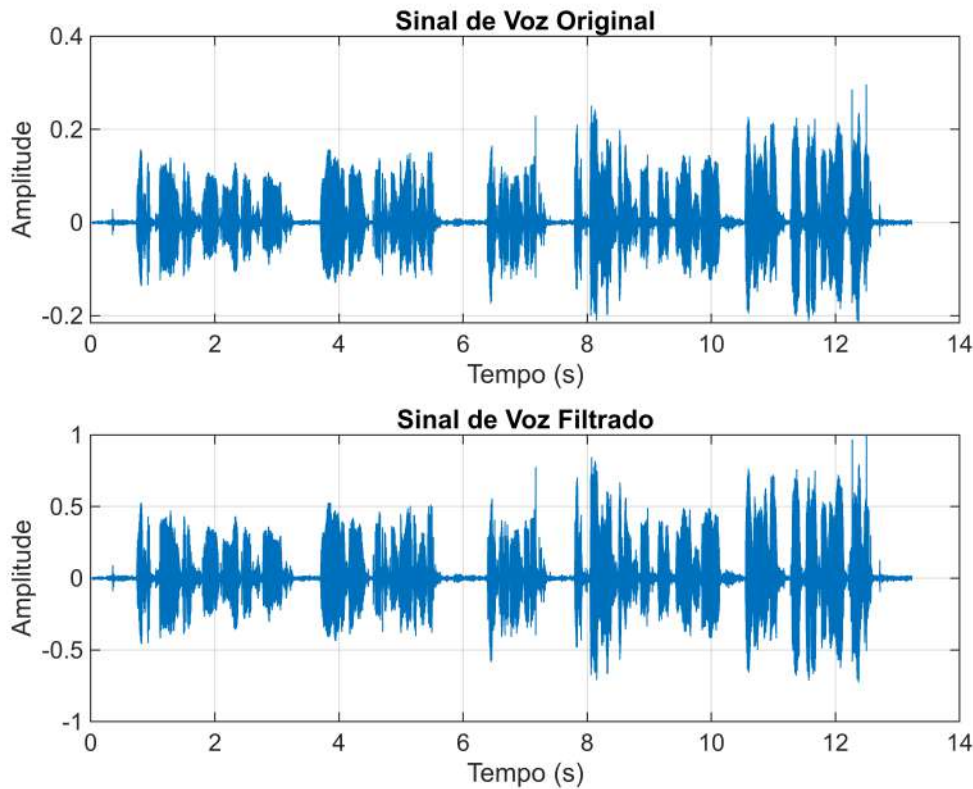
```
tic;  
audio_saida = overlap_add(audio_entrada, h_filtro, N);  
tempo_processamento = toc;  
disp(['Tempo de processamento: ', num2str(tempo_processamento), ' s']);
```

Tempo de processamento: 0.07011 s

```
% Normalizar áudio de saída  
audio_saida = audio_saida / max(abs(audio_saida));  
  
% Salvar áudio de saída  
audiowrite('audio_saida.wav', audio_saida, Fs);  
disp('Áudio processado salvo em: audio_saida.wav');
```

Áudio processado salvo em: audio_saida.wav

```
%% Plotar sinais no tempo  
figure('Name', 'Passo 2 - Sinais no Tempo');  
  
t_entrada = (0:length(audio_entrada)-1) / Fs;  
t_saida = (0:length(audio_saida)-1) / Fs;  
  
subplot(2,1,1);  
plot(t_entrada, audio_entrada);  
title('Sinal de Voz Original');  
xlabel('Tempo (s)'); ylabel('Amplitude');  
grid on;  
  
subplot(2,1,2);  
plot(t_saida, audio_saida);  
title('Sinal de Voz Filtrado');  
xlabel('Tempo (s)'); ylabel('Amplitude');  
grid on;
```



```

%% Plotar sinais na frequência
figure('Name', 'Passo 2 - Sinais na Frequência');

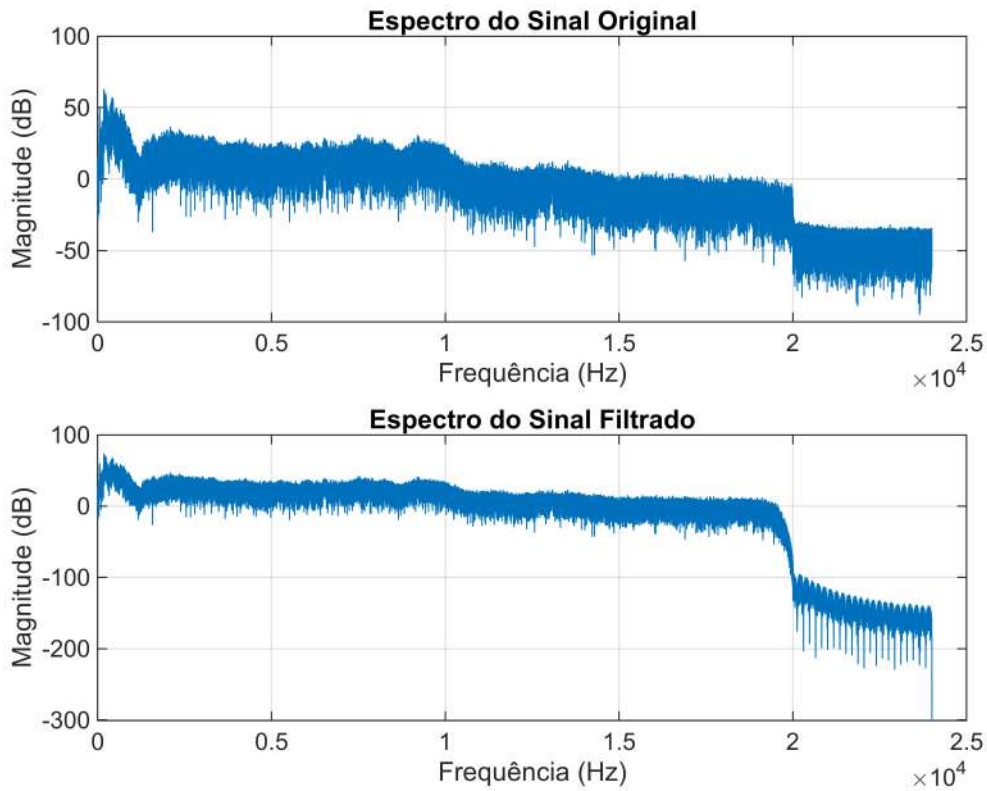
% FFT do sinal de entrada
NFFT = 2^nextpow2(length(audio_entrada));
X = fft(audio_entrada, NFFT);
f_x = Fs/2 * linspace(0, 1, NFFT/2+1);

% FFT do sinal de saída
Y = fft(audio_saida, NFFT);
f_y = Fs/2 * linspace(0, 1, NFFT/2+1);

subplot(2,1,1);
plot(f_x, 20*log10(abs(X(1:NFFT/2+1))));
title('Espectro do Sinal Original');
xlabel('Frequência (Hz)'); ylabel('Magnitude (dB)');
grid on;

subplot(2,1,2);
plot(f_y, 20*log10(abs(Y(1:NFFT/2+1))));
title('Espectro do Sinal Filtrado');
xlabel('Frequência (Hz)'); ylabel('Magnitude (dB)');
grid on;

```



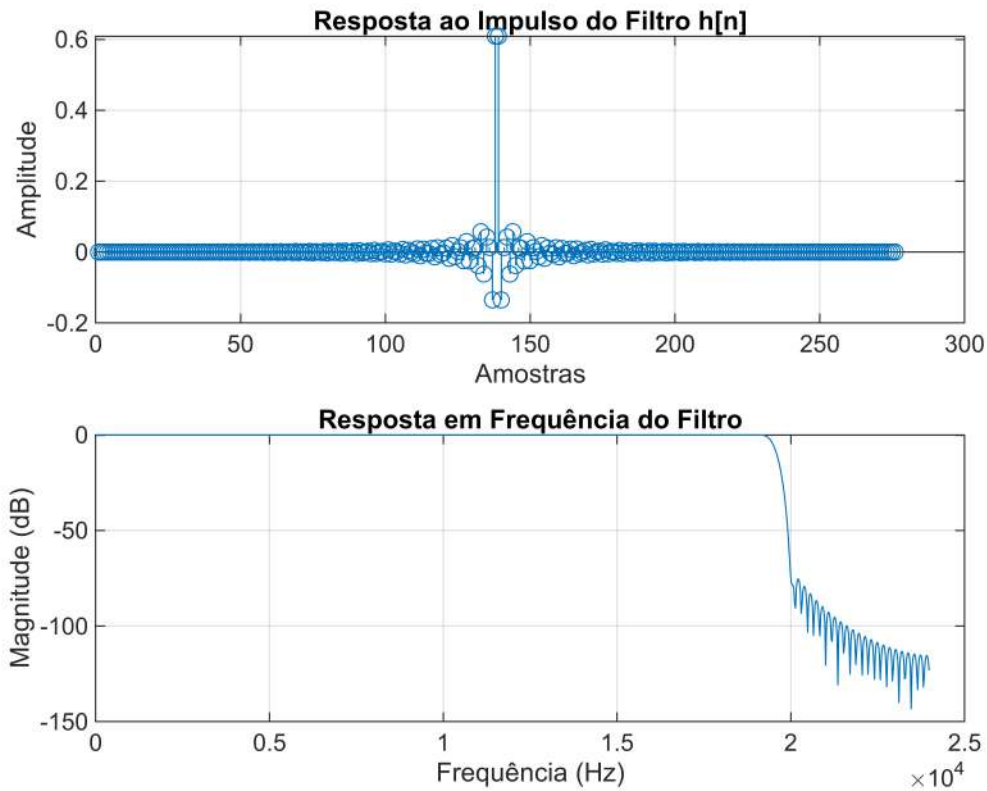
```

%% Plotar resposta do filtro
figure('Name', 'Características do Filtro');

subplot(2,1,1);
stem(h_filtro);
title('Resposta ao Impulso do Filtro h[n]');
xlabel('Amostras'); ylabel('Amplitude');
grid on;

subplot(2,1,2);
[H, f_h] = freqz(h_filtro, 1, 1024, Fs);
plot(f_h, 20*log10(abs(H)));
title('Resposta em Frequência do Filtro');
xlabel('Frequência (Hz)'); ylabel('Magnitude (dB)');
grid on;

```



```
%% Tocar áudios (opcional)
disp(' ');
```

```
disp('Deseja ouvir os áudios? (s/n)');
```

Deseja ouvir os áudios? (s/n)

```
resposta = input('', 's');
if strcmpi(resposta, 's')
    disp('Reproduzindo áudio original...');
    sound(audio_entrada, Fs);
    pause(duracao + 1);

    disp('Reproduzindo áudio filtrado...');
    sound(audio_saida, Fs);
end
```

Reproduzindo áudio original...
Reproduzindo áudio filtrado...

===== PROCESSAMENTO CONCLUÍDO =====

Arquivos gerados:

- audio_entrada.wav
- audio_saida.wav
- Figuras com todos os gráficos

```
%% ===== FUNÇÃO OVERLAP-ADD =====
```

```
function y = overlap_add(x, h, N)
% OVERLAP_ADD - Implementa convolução usando método de Sobreposição e Soma
%
% Entradas:
%   x - Sinal de entrada (vetor linha)
%   h - Resposta ao impulso do filtro (vetor linha)
%   N - Número de pontos da FFT
%
% Saída:
%   y - Sinal de saída (convolução de x com h)

M = length(h);           % Comprimento do filtro
L = N - M + 1;           % Comprimento de cada bloco
Lx = length(x);          % Comprimento do sinal de entrada

% Número de blocos necessários
num_blocos = ceil(Lx / L);

% Zero-padding no sinal de entrada se necessário
x_padded = [x, zeros(1, num_blocos * L - Lx)];

% Zero-padding no filtro para ter comprimento N
h_padded = [h, zeros(1, N - M)];

% FFT do filtro
H = fft(h_padded, N);

% Inicializar saída
y = zeros(1, Lx + M - 1);

% Processar cada bloco
for i = 1:num_blocos
    % Extrair bloco de entrada
    idx_inicio = (i-1) * L + 1;
    idx_fim = min(i * L, length(x_padded));
    bloco = x_padded(idx_inicio:idx_fim);

    % Zero-padding no bloco para ter comprimento N
    bloco_padded = [bloco, zeros(1, N - length(bloco))];

    % FFT do bloco
    X_bloco = fft(bloco_padded, N);
```

```

% Multiplicação no domínio da frequência
Y_bloco = X_bloco .* H;

% IFFT para obter o resultado no tempo
y_bloco = real(ifft(Y_bloco, N));

% Adicionar à saída (overlap-add)
idx_saida_inicio = (i-1) * L + 1;
idx_saida_fim = idx_saida_inicio + N - 1;

% Garantir que não ultrapasse o tamanho da saída
if idx_saida_fim > length(y)
    y = [y, zeros(1, idx_saida_fim - length(y))];
end

y(idx_saida_inicio:idx_saida_fim) = y(idx_saida_inicio:idx_saida_fim) +
y_bloco;
end

% Truncar saída para o tamanho correto
y = y(1:Lx + M - 1);
end

```


Relatório da Simulação Computacional # 5

Descentes: Maria Thieza Melo

Thiele Gabriel Gomes Costa

Hanna Vitória

1. Metodologia e Parâmetros do Projeto

O método escolhido para a convolução por bloco utilizando a DFT (Transformada Discreta de Fourier) foi o de sobreposição e soma (Overlap - Add).

A eficiência do método depende da relação entre os seguintes parâmetros:

- comprimento de filtro (M);
- tamanho de FFT (N);
- comprimento de bloco útil (L).

2. Passo 1: Validação de convolução (simulação # 1)

O passo 1 validar a implementação do algoritmo Overlap - Add comparando seu resultado com a função de convolução padrão (conv.).

2. 1. Geração de Sinais de Teste

Sinal de Entrada ($x[n]$): O sinal foi gerado com amostras aleatórias de comprimento $N_x = 5550$. Este valor foi escolhido para ser aproximadamente $15,3 \cdot L$ ($15,3 \cdot 363 \approx 5550,9$) e, crucialmente, não múltiplo de L , garantindo que o algoritmo trate corretamente o bloco final.

Filtro ($h[n]$): Gerado com $M = 150$ amostras aleatórias.

2. 2. Resultados e Análise de Erro

A implementação do Overlap - Add foi executada e comparada ao sinal de referência $y_{conv} = x[n] * h[n]$.

Gráfico de comparação: A saída do Overlap - Add se sobrepõe perfeitamente à saída de convolução padrão. O Erro Quadrático Médio (MSE) calculado entre as 2 saídas foi tipicamente inferior a 10^{-20} .

Conclusão da Validação: O MSE calculado foi muito baixo, o que confirma que o algoritmo Overlap - Add implementado é funcionalmente equivalente à convolução no tempo e está pronto para ser utilizado na simulação 2.

3. Passo 2: Aplicação em Filtragem de Voz (Simulação #2)

O algoritmo overlap-add validado foi utilizado para substituir filter na filtragem de sinal de voz, confirmando a aplicação do método na prática.

3.1. Sinais de Entrada e Filtro

Sinal de Entrada ($x[n]$): Uma amostra de voz de mínimo 10 segundos foi gravada.

Filtro ($h[n]$): Foi utilizado o mesmo filtro FIR projetado no projeto anterior de UI (Unidade de Instrução Integrada).

3.2. Resultados no Domínio do Tempo e Frequência

Gráficos no Domínio do Tempo:

A forma de onda do sinal de saída $y[n]$ é visivelmente mais suave do que o sinal de entrada $x[n]$ (sinal de voz corrompido), indicando que as componentes de alta frequência (ruído) foram atenuadas.

Observa-se um pequeno atraso de $(N-1)/2$ amostras no sinal de saída devido à fase linear do filtro FIR.

Gráficos no Domínio da Frequência:

O espectro de entrada $|X(f)|$ exibe picos de ruído (ex.: em 5 kHz) que o filtro FIR foi projetado para rejeitar.

O Espectro de saída $|Y(f)|$ demonstra que a energia nessas frequências de ruído foi drasticamente reduzida (atenuação). O conteúdo espectral na banda de passagem do filtro (frequências de voz) foi preservado, validando o propósito do filtro e a correta aplicação do algoritmo overlap-add na filtragem real.

Conclusão Final: O projeto demonstrou com sucesso a implementação eficiente de convolução por bloco (overlap-add) e sua aplicabilidade direta em sistemas de PSD, substituindo o algoritmo padrão de convolução no tempo por um método otimizado para a arquitetura FFT.